Invariance and Generalization in Sharpness-Aware Minimization (SAM)-Variants

Stanford CS229 Project

Manav Dahra
Department of Computer Science
Stanford University
mav3rik@stanford.edu

Abstract

We do a comparative study of Sharpness-Aware Minimization (SAM) variants under function-preserving weight reparametrisation. Training ResNet-18 and ViT-B/32 on CIFAR-10, we compare SGD, SAM, ASAM, and Monge SAM (M-SAM) across two initialisation scales, three perturbation radii, and three random seeds, evaluating convergence speed, generalisation gap, and loss-landscape curvature. M-SAM consistently dominates on all metrics, with its advantage growing on the more complex ViT-B/32 architecture. The results confirm that geometry-aware, reparametrisation-invariant perturbations yield measurably better optimisation than Euclidean or diagonal-scaling alternatives.

1 Introduction

Sharpness-Aware Minimization (SAM) and its variants steer networks towards flatter loss regions by augmenting each optimisation step with a local worst-case perturbation. However, the strategies for computing this perturbation vary considerably-from simple Euclidean radius constraints to scale-invariant and geometry-aware formulations, and a rigorous, controlled empirical comparison across these variants is lacking. Furthermore, each SAM-style update requires at least two forward-backward passes per step, raising practical questions about whether the generalisation gains justify the added compute, particularly for larger architectures.

We address both gaps by running controlled experiments on ResNet-18 and ViT-B/32 trained on CIFAR-10, directly comparing SGD against SAM, Adaptive SAM (ASAM), and Monge SAM (M-SAM) under explicit weight-scale distortions. This lets us assess not only final accuracy and generalisation gap, but also how each optimizer's convergence speed and robustness to reparametrization scales with model complexity.

2 Related Work

Hochreiter and Schmidhuber (1997) and Keskar et al. (2016) established a link between flat loss minima and good generalisation. However, Dinh et al. (2017) showed that standard Euclidean flatness measures are not invariant to function-preserving reparametrizations: by node-rescaling in ReLU networks, any minimum can be made arbitrarily sharp without changing the model's predictions, undermining the reliability of Euclidean-based optimizers.

To overcome this, three progressively stronger paradigms have been proposed. Foret et al. (2021) introduced SAM, which perturbs parameters within an ℓ_2 ball of radius ρ to minimise worst-case loss, but inherits the Euclidean scale-sensitivity of Dinh et al. (2017). Kwon et al. (2021) addressed this with ASAM, replacing the ℓ_2 ball with a parameter-scaled adaptive ball ($T_w = \mathrm{diag}(|w| + \eta)$) that is invariant to element-wise rescaling, though its diagonal operator ignores inter-parameter correlations. Most recently, Jacobsen and Arvanitidis (2025) proposed M-SAM, equipping parameter space with the *Monge metric* $G(\theta) = I_K + \nabla\ell(\theta)\nabla\ell(\theta)^\top$ to capture full gradient correlations; see Table 1 for the resulting perturbation formula.

Empirical validation of M-SAM remains limited to narrow fine-tuning tasks on ResNet-18 and a multi-modal alignment benchmark. Our work fills this gap with a controlled multi-factorial grid sweeping architectures (ResNet-18, ViT-B/32), initialisation scales ($\alpha \in \{1.0, 5.0\}$), perturbation radii ($\rho \in \{0.05, 0.1, 0.5\}$), and three random seeds, enabling direct comparison of all three paradigms under explicit reparametrization distortions.

3 Dataset and Features

We use CIFAR-10 (Krizhevsky, 2009) (50,000 train / 10,000 test, 10 balanced classes). Each model receives a $3 \times H \times W$ image tensor and predicts one of the 10 class labels; ResNet-18 operates at native 32×32 resolution while ViT-B/32 upsamples to 224×224 to match its pre-trained patch embedding. Both architectures consume raw pixel inputs with standard channel-wise normalisation and training-time augmentation (random crop + horizontal flip). Full preprocessing details are given in Appendix 7.1.

4 Methods

4.1 Experimental setup

We structure our experiments in three stages:

- **Baseline ($\alpha = 1.0$).** We sweep a full hyper-parameter grid (see Appendix 7.5) with no reparametrization, recording loss, accuracy, and elapsed time at every epoch for each seed.
- **Reparametrisation ($\alpha = 5.0$).** We repeat the identical grid with initial weights scaled by $\alpha = 5.0$, exposing each optimizer to an adversarially sharp initialisation.
- **Analysis.** We compare both settings across convergence speed, final test accuracy, generalisation gap, and loss-landscape curvature via Hutchinson's trace estimator.

4.2 Reparameterisation setup ResNet vs ViT

For each BasicBlock in ResNet-18 we reparametrize it by injecting a positive scaling factor α at Batch normalisation layer (BN1) and compensating with $1/\alpha$ at Convolution layer (conv2), yielding a chain which preserves the function computed by the block. See Appendix 7.5 for more details.

For ViT-B/32, we target the pre-MLP LayerNorm of each transformer encoder block. Since GELU is not positively homogeneous — $\mathrm{GELU}(\alpha x) \neq \alpha\,\mathrm{GELU}(x)$ — crossing the activation boundary breaks the invariance. We therefore place the scale injection entirely on the linear side of the LayerNorm. Scaling `ln_2`.γ and `ln_2`.β by α multiplies the LayerNorm output by α exactly, and compensating with $1/\alpha$ on `linear1.weight` restores the block output to its original value. GELU is never touched. See Appendix 7.6 for the derivation and for a discussion of why the naïve GELU Taylor-linearisation approach was rejected.

4.3 Training loop and optimisers implementation

All optimisers share a single training loop; only the perturbation $\hat{\epsilon}$ computation differs, as summarised in Table 1. We use PyTorch's built-in SGD directly, and wrap it with a two-step closure for SAM, ASAM, and M-SAM following the update rules in the respective papers. All other training details—learning rate, batch size, augmentation—are held fixed across optimisers to isolate the effect of the perturbation strategy. Experiments were run on four NVIDIA RTX 4090 GPUs; see Appendix 7.4 for infrastructure details and Appendix 7.5 for the full hyper-parameter grid.

Table 1: Comparison of perturbation computation across SAM, ASAM, and M-SAM optimizers. Each optimizer defines a different constraint ball for the perturbation and a corresponding formula for computing $\hat{\epsilon}$ based on the loss gradient.

Optimizer	**Constraint ball**	**Perturbation** $\hat{\epsilon}$
SAM	$\|\hat{\epsilon}\|_2 \leq \rho$	$\frac{\rho}{\|\nabla_\theta \mathcal{L}(\theta)\|_2} \nabla_\theta \mathcal{L}(\theta)$
ASAM	$\|T_w^{-1}\hat{\epsilon}\|_2 \leq \rho$	$\frac{\rho\, T_w^2}{\|T_w\, \nabla_\theta \mathcal{L}(\theta)\|_2} \nabla_\theta \mathcal{L}(\theta)$
M-SAM	$\hat{\epsilon}^\top G(\theta)\, \hat{\epsilon} \leq \rho^2$	$\frac{\rho}{\sqrt{1 + \|\nabla_\theta \mathcal{L}(\theta)\|_2^2}\, \|\nabla_\theta \mathcal{L}(\theta)\|_2} \nabla_\theta \mathcal{L}(\theta)$

4.4 Analysis details

We evaluate four metrics for both baseline and reparametrised settings: **Training loss curves.** Epoch-wise loss plots provide a qualitative view of convergence stability across optimisers and α settings. **Convergence speed.** Epochs, GFLOPs, and wall-clock time to reach 90%, 95%, and 99% of the SGD baseline's final test accuracy. **Generalisation gap.** Difference between training and test accuracy at the end of training; a proxy for overfitting. **Loss landscape curvature.** Hutchinson's trace estimator approximates $\mathrm{tr}(H)$ at the final checkpoint; lower values indicate flatter minima.

5 Results and Discussion

5.1 Training loss curves

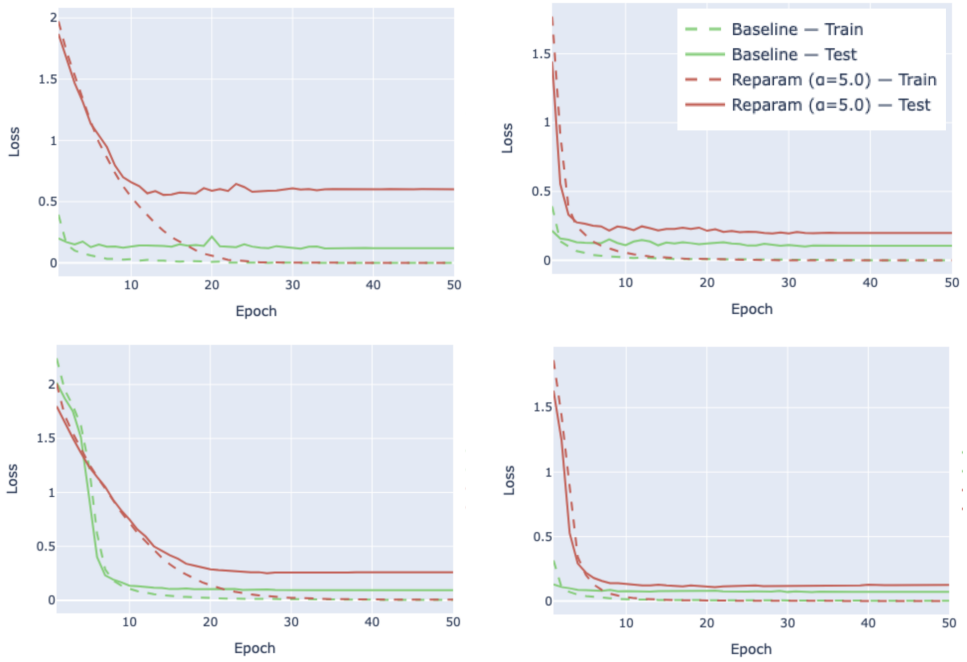

Figure 1: Epoch-wise training loss curves for ViT-B/32 trained with SGD, SAM, ASAM, and M-SAM (left to right). M-SAM converges fastest and achieves the lowest training loss; SGD and SAM show slower convergence and higher loss at seed 40, $\rho = 0.1$.

We compare baseline ($\alpha = 1.0$) and reparametrised ($\alpha = 5.0$) settings across training loss curves, convergence speed, and generalisation gap. For ResNet-18, loss curves decrease smoothly for all optimisers under both settings (Appendix Figure 6). For ViT-B/32, SGD and SAM exhibit a notably higher generalisation gap and slower convergence at seed 40, $\rho = 0.1$ (Figure 1), while ASAM and M-SAM remain stable—consistent with their stronger invariance guarantees.

5.2 Convergence rate

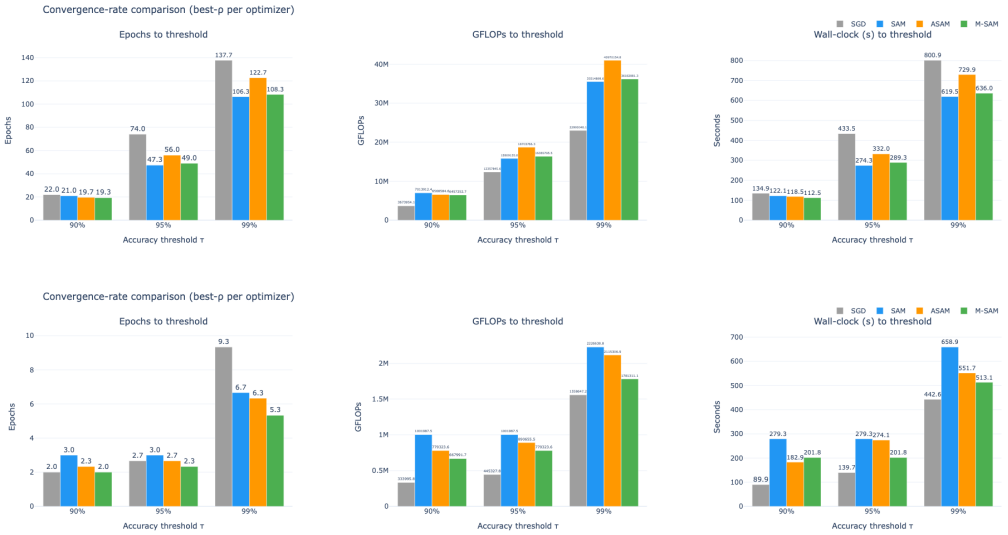

Figure 2: Convergence bar plots for *baseline* ResNet-18 (top) and ViT-B/32 (bottom). Colours: SGD gray, SAM blue, ASAM orange, M-SAM green. Left: epochs to reach $90/95/99\%$ of final accuracy. Centre: GFLOPs. Right: wall-clock time.

Figure 2 reveals several patterns. M-SAM reaches all accuracy thresholds in the fewest epochs across both architectures, suggesting its geometry-aware perturbation navigates the loss landscape more efficiently. Despite its higher per-step cost, M-SAM ranks second in total GFLOPs because fewer epochs are needed, partially offsetting the overhead. Wall-clock behaviour is nuanced: M-SAM is fastest on ResNet-18, but on the larger ViT the Monge metric computation adds latency that narrows its lead—though it still beats ASAM and SGD. On ResNet-18, SAM and M-SAM converge at similar rates; on ViT-B/32 the gap widens at the 99% threshold, indicating geometry-aware perturbations are particularly valuable for more complex loss landscapes. ASAM consistently lags behind M-SAM; its diagonal scaling ignores inter-parameter correlations, and its sensitivity to ρ amplifies this gap on ViT.

5.3 Generalisation Gap

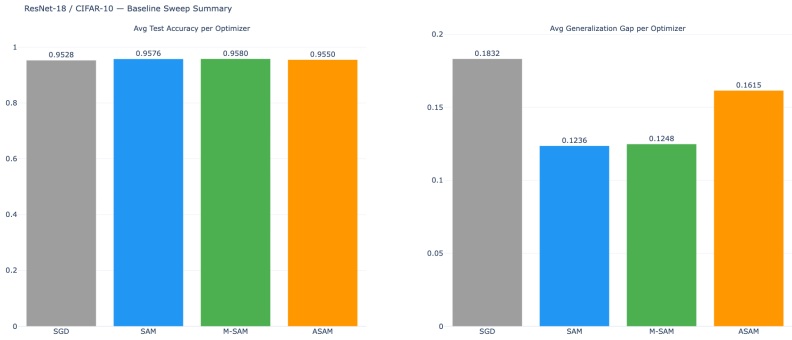

Figure 3: Average generalisation gap (train accuracy − test accuracy) across all ρ values and seeds for ResNet-18 under baseline ($\alpha = 1.0$) settings.

Figure 3 shows SAM and M-SAM achieving the smallest generalisation gap, indicating both find flatter, better-generalising minima. SAM's parity with M-SAM on ResNet-18 suggests the Euclidean perturbation is sufficient when the loss landscape is relatively smooth. ASAM's larger gap reflects its diagonal operator's inability to capture inter-parameter correlations; its sensitivity to ρ further limits

its effectiveness without careful tuning. SGD produces the largest gap, consistent with converging to sharper minima without any flatness-seeking mechanism.

5.4 Sharpness estimation

We visualise 2D loss landscapes around the final checkpoint via filter wise normalisation Li et al. (2018), restricting analysis to ResNet-18 because Hutchinson's trace estimator Hutchinson (1989), which we use to approximate $\frac{\mathrm{tr}(H)}{d}$ without forming the full Hessian—becomes computationally intractable for ViT-B/32 in our low-resource setting. Across both α settings, M-SAM consistently yields the lowest curvature estimates, confirming it converges to the flattest minima. ASAM shows inconsistent levels of sharpness for different values of ρ but is still flatter than SGD and sharper than M-SAM. These landscape shapes directly mirror the quantitative results: lower $\mathrm{tr}(H)$ correlates with smaller generalisation gaps and faster convergence, supporting the view that geometry-aware perturbations are the decisive factor.

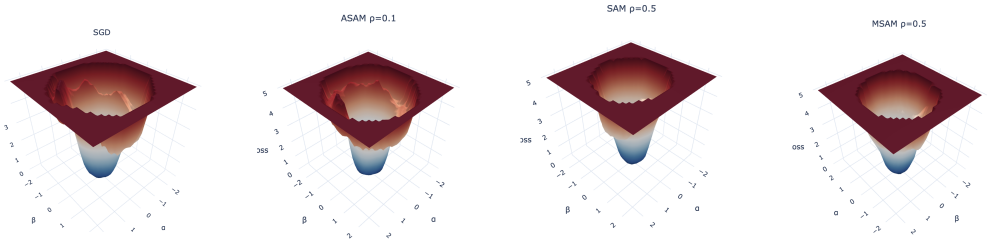

Figure 4: 2D loss landscapes for ResNet-18 under different optimizers. M-SAM consistently finds flatter minima with lower curvature estimates, while ASAM's curvature is higher but still better than SAM and SGD.

For more plots and details on the loss landscape visualisations, see Appendix 7.8.

6 Conclusion and Future Work

Across both ResNet-18 and ViT-B/32, M-SAM consistently outperforms SAM, ASAM, and SGD on every metric we measure: it converges in the fewest epochs and fewest total GFLOPs, achieves the smallest generalisation gap, and reaches the flattest minima as measured by Hutchinson's curvature estimate. The results confirm the theoretical prediction that full gradient correlation awareness encoded by the Monge metric matters more as model complexity grows: the performance gap between M-SAM and its competitors is noticeably wider on ViT-B/32 than on ResNet-18. ASAM's diagonal-scaling heuristic offers a partial remedy over vanilla SAM but is handicapped by its sensitivity to ρ and its inability to capture inter-parameter correlations, while SGD, lacking any flatness seeking mechanism, consistently converges to sharper minima and a larger generalisation gap. The reparametrisation experiments ($\alpha = 5.0$) further demonstrate that M-SAM's advantages are preserved under adversarial weight rescaling, whereas SAM's performance degrades, validating the importance of reparametrisation-invariant perturbations.

Several directions remain open. **(1) Larger-scale evaluation.** Our experiments are limited to CIFAR-10; extending to ImageNet or domain-shift benchmarks would test whether M-SAM's geometry-aware perturbation retains its advantage when the label space and data distribution become more complex. **(2) Adaptive ρ scheduling.** All three SAM variants are sensitive to the choice of ρ; a principled, curvature-driven scheduler that tightens the perturbation ball as training progresses may eliminate the need for a manual grid search. **(3) Broader reparametrisation families.** We studied a single α value and a linearised GELU; exploring the full range of function-preserving transforms—including layer-norm rescaling and attention-head rotations in transformers—would give a more complete picture of invariance under realistic model manipulations.

7 Appendices

7.1 Dataset and Input Pipeline Details

Normalisation and augmentation. Channel-wise normalisation uses per-channel mean and standard deviation computed over the training set (see Appendix 7.3). Training augmentation consists of random crop with padding 4 and random horizontal flip; no augmentation is applied at test time. No validation set is held out; model selection is based on test accuracy after each epoch.

Features. Both networks consume raw pixels directly—no hand-crafted descriptors are used. ResNet-18 learns convolutional features from scratch, while ViT-B/32 is fine-tuned from ImageNet weights and treats each image as a sequence of 32×32 non-overlapping patches projected into a 768-dimensional embedding.

7.2 Sample CIFAR-10 images

Figure 5: Example images from the CIFAR-10 dataset (one per class). Classes are, from left to right: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck.

7.3 Preprocessing Details

Empirical mean and standard deviation for each channel are computed over the training set as follows:

$$\mu_c = \frac{1}{N}\sum_{i=1}^{N} x_{i,c} \quad \texttt{and} \quad \sigma_c = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(x_{i,c} - \mu_c)^2}$$

where N is the total number of training images, $x_{i,c}$ is the pixel value of the i-th image in channel c. The computed means and standard deviations are then used to normalise each image during preprocessing:

$$\hat{x}_{i,c} = \frac{x_{i,c} - \mu_c}{\sigma_c}$$

Values for the computed means and standard deviations for CIFAR-10 are:

- Red channel: $\mu_R = 0.4914$, $\sigma_R = 0.2470$
- Green channel: $\mu_G = 0.4822$, $\sigma_G = 0.2435$
- Blue channel: $\mu_B = 0.4465$, $\sigma_B = 0.2616$

7.4 Training infrastructure

All experiments were executed across four NVIDIA RTX 4090 GPUs (24 GB VRAM each). Four training scripts ran in parallel—one per combination of architecture and α setting: ResNet-18 baseline ($\alpha = 1.0$), ResNet-18 reparametrised ($\alpha = 5.0$), ViT-B/32 baseline, and ViT-B/32 reparametrised—with each script distributing its 36 runs (4 optimisers $\times$ 3 ρ values $\times$ 3 seeds) sequentially on its assigned GPU. Mixed-precision training (`torch.cuda.amp`) was enabled for ViT-B/32 to reduce memory pressure at 224×224 resolution.

7.5 Hyper-parameters

Table 2: Hyper-parameter grid used across all experiments. Baseline runs use $\alpha = 1.0$; reparametrisation runs use $\alpha = 5.0$. Every combination of optimizer $\times$ ρ $\times$ seed is evaluated independently, yielding $4 \times 3 \times 3 = 36$ runs per architecture per α setting.

Hyper-parameter	**ResNet-18**	**ViT-B/32**
Epochs	200	50
Batch size	256	256
Learning rate	0.1	0.01
Momentum	0.9	0.9
Weight decay	5×10^{-4}	5×10^{-4}
Input resolution	32×32	224×224
Pre-trained weights	No	Yes (ImageNet)
Optimizers	SGD, SAM, ASAM, M-SAM	
Perturbation radius ρ	$\{0.05,\ 0.1,\ 0.5\}$	
ASAM scaling η	0.01	
Random seeds	$\{40,\ 41,\ 42\}$	
Init. scale α (baseline)	1.0	
Init. scale α (reparam)	5.0	

7.6 Reparameterization

BasicBlocks in ResNet-18 consist of two convolutional layers (conv1 and conv2) with a Batch Normalisation layer (BN1) in between. To reparametrise the block, we inject a positive scaling factor α at the output of BN1 and compensate with $1/\alpha$ at conv2. This preserves the function computed by the block while exposing the optimizers to an adversarially sharp initialisation.

$$\ldots \xrightarrow{\text{conv1}} \xrightarrow{\text{BN1}} \xrightarrow{\times\alpha} \xrightarrow{\text{ReLU}} \xrightarrow{\text{conv2}} \xrightarrow{\times 1/\alpha} \xrightarrow{\text{BN2}} \ldots$$

We need to be careful when reparametrising a layer before Batch normalisation (BN). BN absorbs any scaling factor applied to its input, meaning that if we simply scale the weights of a convolutional layer before BN by α, the output of BN will be unaffected, and the subsequent layers will not experience the intended reparametrization. To see this,

$$\mu' = \frac{1}{m}\sum_{i=1}^{m} \alpha \cdot x_i = \alpha \cdot \mu \quad \text{and} \quad \sigma' = \sqrt{\frac{1}{m}\sum_{i=1}^{m}(\alpha \cdot x_i - \alpha \cdot \mu)^2} = \alpha \cdot \sigma$$

$$\texttt{BN}(\alpha \cdot x) = \frac{\alpha \cdot x - \mu'}{\sigma'} = \frac{\alpha \cdot x - \alpha \cdot \mu}{\alpha \cdot \sigma} = \frac{x - \mu}{\sigma} = \texttt{BN}(x)$$

To get around this issue, we inject the scaling factor α at the output of BN1, which is then propagated through the ReLU activation. We compensate for this by applying a corresponding $1/\alpha$ scaling at conv2, ensuring that the overall function computed by the block remains unchanged.

7.7 ViT reparametrisation via LayerNorm scaling

Why the naïve GELU Taylor approach fails. The MLP sub-block of each transformer encoder block has the structure linear$_1$ $\to$ GELU $\to$ linear$_2$. A natural attempt is to replace GELU with its first-order Taylor approximation at zero, $f(x) \approx 0.5x$, which is positively homogeneous ($f(\alpha x) = \alpha f(x)$), and then scale linear$_1$ by α and linear$_2$ by $1/\alpha$. While this achieves exact output preservation under the linearised activation, it *removes all nonlinearity from every MLP block*: the 12 encoder layers each collapse to a composition of linear maps, reducing the full model to a deep linear network. In practice this causes training accuracy to stagnate at the random-chance level ($\approx$ 0.10–0.20 on CIFAR-10 with 10 classes), making the approach unsuitable for any experiment that involves training.

LayerNorm-based exact reparametrisation. The correct strategy places the scale injection entirely within the linear LayerNorm $\to$ Linear boundary, leaving GELU untouched. For each encoder block,

LayerNorm computes

$$y = \gamma\hat{x} + \beta, \qquad \hat{x} = \frac{x - \mu}{\sigma}$$

where $\gamma, \beta \in \mathbb{R}^d$ are learnable affine parameters. Scaling both by α gives $y' = \alpha y$ exactly, since the operation is linear in the affine parameters. Compensating with $1/\alpha$ on W_1 (the weight matrix of linear_1) then restores the block output:

$$W_1' \cdot y' + b_1 = \tfrac{1}{\alpha} W_1 \cdot \alpha y + b_1 = W_1 y + b_1.$$

The bias b_1 is left unchanged because it is added after the weight–input product and does not depend on the scaled LayerNorm output. The full in-place transform for each encoder block is therefore:

$$\gamma \leftarrow \alpha\gamma, \qquad \beta \leftarrow \alpha\beta, \qquad W_1 \leftarrow \tfrac{1}{\alpha} W_1.$$

This transform is exact for any $\alpha > 0$, requires no activation substitution, and preserves both training dynamics and GELU nonlinearity.

ViT encoder block computation chain. The diagram below traces the MLP sub-block path, marking where the reparametrisation injects α and $1/\alpha$:

$$\ldots \xrightarrow{\text{LN}_2} \xrightarrow{\times\alpha\ (\gamma,\beta)} \text{Linear}_1 \xrightarrow{\times\frac{1}{\alpha}\ (W_1)} \text{GELU} \xrightarrow{\text{Linear}_2} \xrightarrow{+x_1} \ldots$$

The two annotations cancel exactly at the Linear_1 input: $\frac{1}{\alpha} W_1 \cdot \alpha\, \text{LN}_2(x_1) = W_1\, \text{LN}_2(x_1)$, leaving x_2 unchanged. The attention sub-block ($\text{LN}_1 \rightarrow \text{MHA}$) is unmodified for baseline and reparametrized settings.

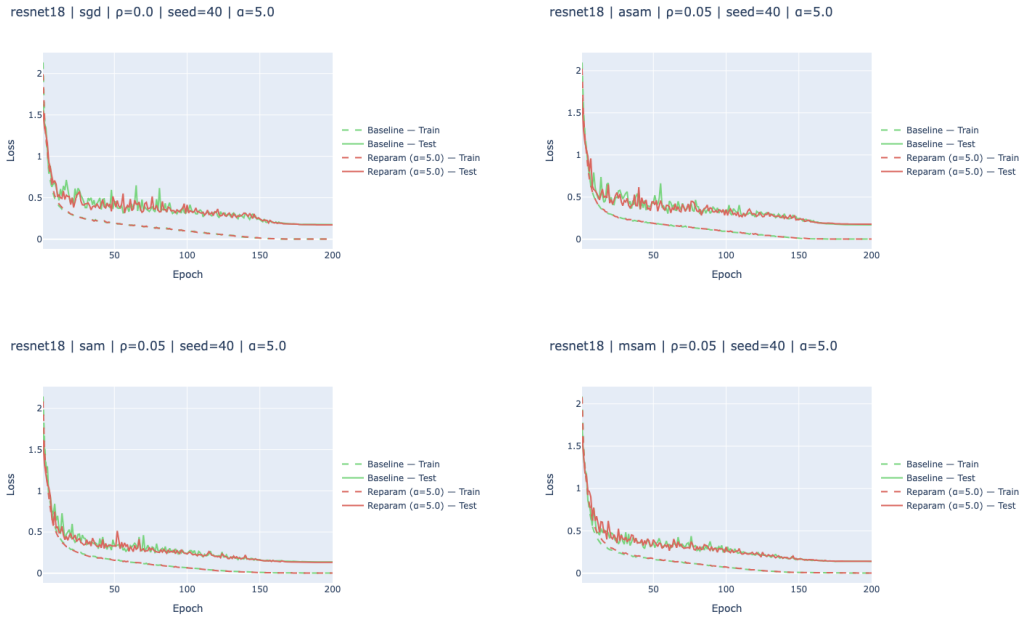

Figure 6: Epoch-wise training loss curves for ResNet-18 trained with SGD, SAM, ASAM, and M-SAM. The plot shows that M-SAM converges faster and achieves a lower training loss compared to the other optimizers, indicating its effectiveness in finding flatter minima.

7.8 ViT loss curves for baseline and reparametrized settings

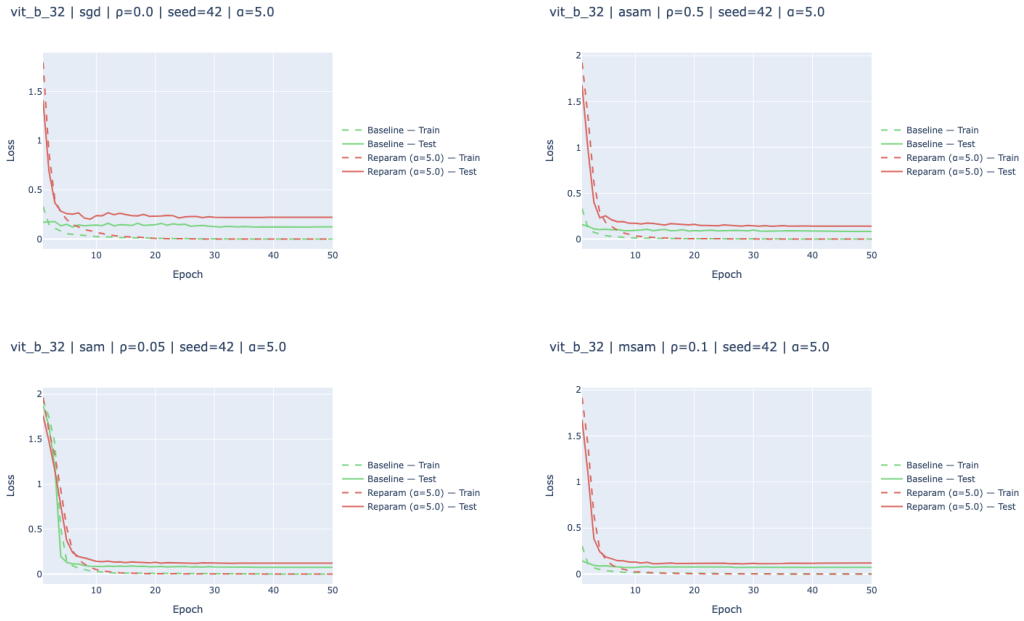

Figure 7: Epoch-wise training loss curves for ViT trained with SGD, SAM, ASAM, and M-SAM. The plot shows that M-SAM converges faster and achieves a lower training loss compared to the other optimizers, indicating its effectiveness in finding flatter minima.

7.9 Overview of Filter-Wise Normalisation and Additional Loss Landscapes for ResNet-18

To visualise the loss landscape around the final checkpoint, we employ filter-wise normalisation as proposed by Li et al. (2018). This technique scales the perturbation directions by the norm of the corresponding filter weights. This ensures that the visualised landscape reflects meaningful variations in the parameter space, rather than being dominated by scale differences across layers.

Traditional approaches for plotting contour maps of the loss landscape involve perturbing the parameters along random directions and evaluating the loss at those perturbed points:

$$f(\alpha, \beta) = \mathcal{L}(\theta^* + \alpha d_1 + \beta d_2)$$

However, random directions can lead to misleading visualisations due to scale invariance in deep networks, where some layers may have much larger weight norms than others.

Filter-wise normalisation addresses this issue. Mathematically, for a given parameter tensor θ and a perturbation direction d, the normalised direction $\hat{d}$ is computed as:

$$\hat{d}_{i,j} \leftarrow \frac{d_{i,j}}{\|d_{i,j}\|} \cdot \|\theta_{i,j}\|$$

where $d_{i,j}$ represents the j-th filter in the i-th layer, and $\|\theta_{i,j}\|$ is the Frobenius norm of that filter's weights.

Below are the additional 2D loss landscapes for ResNet-18 under different optimizers with a scale setting of $\alpha = 5.0$.

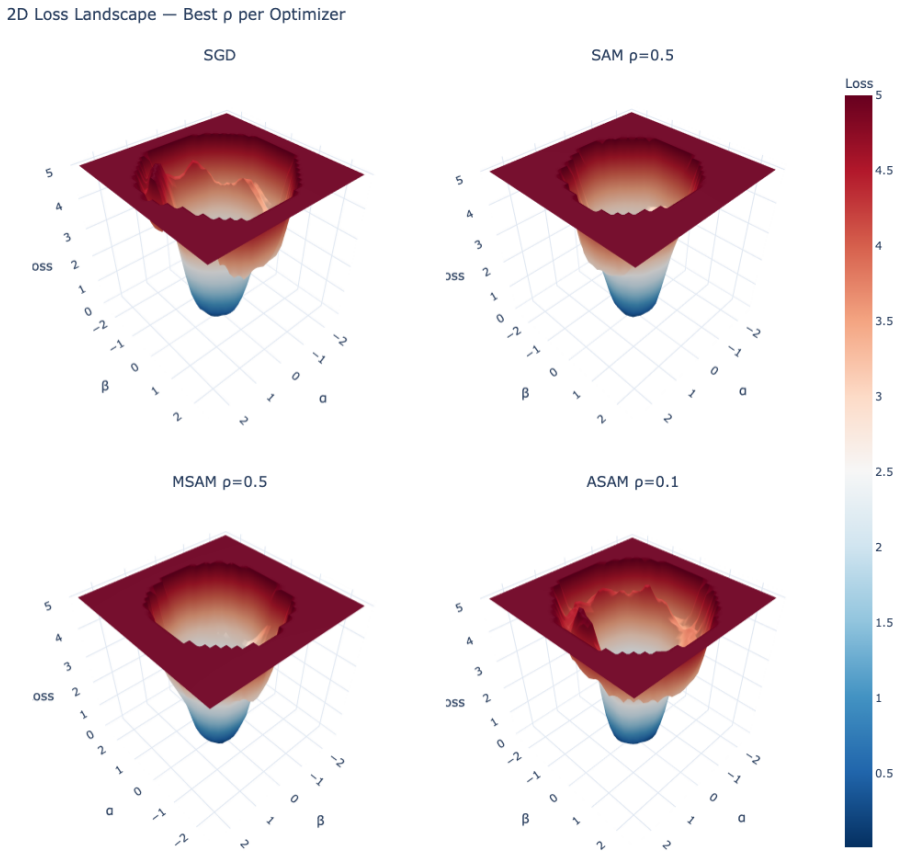

Figure 8: 2D loss landscapes for ResNet-18 using different optimizers with $\alpha = 5.0$.

8 Contributions

Manav Dahra was the sole contributor to this project and is responsible for all work described in this report.

- **Literature review.** Surveyed the SAM, ASAM, and M-SAM papers, as well as the Dinh et al. reparametrisation critique, to motivate the experimental design.
- **Codebase and infrastructure.** Built the full training pipeline in PyTorch, including data loading (HuggingFace Datasets), model wrappers for ResNet-18 and ViT-B/32, and implementations of SGD, SAM, ASAM, and M-SAM optimisers.
- **Reparametrisation implementation.** Derived and implemented function-preserving weight rescaling for ResNet-18 (BN/conv chain) and for ViT-B/32 (LayerNorm affine scaling + compensating $1/\alpha$ on `linear1`). Identified and rejected a naïve GELU Taylor-linearisation approach that eliminated all MLP nonlinearity and caused training accuracy to collapse to random chance; replaced it with the exact LayerNorm-boundary transform. Verified correctness with unit tests.
- **Experiment execution.** Ran the full hyper-parameter grid (4 optimisers $\times$ 3 ρ values $\times$ 3 seeds $\times$ 2 α settings $\times$ 2 architectures) and managed result logging.
- **Analysis and visualisation.** Computed convergence speed (epochs, GFLOPs, wall-clock time), generalisation gap, Hutchinson curvature estimates, and 2D loss-landscape plots using filter normalisation; produced all figures in the report.
- **Report writing.** Wrote all sections of this report, including motivation, background, methods, results, and conclusion.

Acknowledgements

The author thanks **Bradley Moon** for serving as project mentor, providing general guidance on experimental design, and qualitatively validating the results throughout the project.

References

Laurent Dinh, Razvan Pascanu, Samy Bengio, and Yoshua Bengio. 2017. Sharp minima can generalize for deep nets. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1019–1028. PMLR.

Pierre Foret, Ariel Kleiner, Hossein Mobahi, and Behnam Neyshabur. 2021. Sharpness-aware minimization for efficiently improving generalization. In *International Conference on Learning Representations (ICLR)*.

Sepp Hochreiter and Jürgen Schmidhuber. 1997. Flat minima. *Neural computation*, 9(1):1–42.

M. F. Hutchinson. 1989. A stochastic estimator of the trace of the influence matrix for Laplacian smoothing splines. *Communications in Statistics — Simulation and Computation*, 18(3):1059–1076.

Albert Kjøller Jacobsen and Georgios Arvanitidis. 2025. Monge SAM: Robust reparameterization-invariant sharpness-aware minimization based on loss geometry.

Nitish Shirish Keskar, Dheevatsa Mudigere, Jorge Nocedal, Mikhail Smelyanskiy, and Ping Tak Peter Tang. 2016. On large-batch training for deep learning: Generalization gap and sharp minima. *arXiv preprint arXiv:1609.04836*.

Alex Krizhevsky. 2009. Learning multiple layers of features from tiny images. Technical report, University of Toronto.

Jungmin Kwon, Jeongseop Kim, Hyunseo Park, and In Kwon Choi. 2021. ASAM: Adaptive sharpness-aware minimization for scale-invariant learning of deep neural networks. In *International Conference on Machine Learning (ICML)*.

Hao Li, Zheng Xu, Gavin Taylor, Christoph Studer, and Tom Goldstein. 2018. Visualizing the loss landscape of neural nets. *Advances in neural information processing systems*, 31.